

ระบบจองห้องพัก

กลุ่ม Tigger The Goat

รายวิชา 310-2209 Database Systems (ระบบฐานข้อมูล)

สาขาวิชาวิศวกรรมคอมพิวเตอร์ คณะเทคโนโลยีดิจิทัล

สถาบันเทคโนโลยีจิตรลดา

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

อาจารย์ผู้สอน รองศาสตราจารย์ ดร. อานนท์ รุ่งสว่าง

สมาชิกในทีม

ลำดับ	รูปถ่าย	ชื่อ - สกุล	รหัสนักศึกษา	บทบาท
1		นายชีวาวิชญ์ หนูทอง	**6810301003**	**หัวหน้าทีม**
2		นางสาวนรภร มุตติศานต์	**6810301002**	สมาชิก
3		นายกมล ดีเยี่ยงอย่าง	**6810301004**	สมาชิก
4		นายนฤชิต บุญยัง	**6810301012**	สมาชิก
5		นายนันทกรณ์ อำพนันต์	**6810301037**	สมาชิก
6		นายณัฐดนัย สายสกุล	**6810301039**	สมาชิก

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา 310-2209 Database Systems (ระบบฐานข้อมูล) จัดทำขึ้นเพื่อสรุปผลการดำเนินงานของกลุ่ม Tigger The Goat ในหัวข้อ "ระบบจองห้องพัก" โดยโครงงานนี้ออกแบบและสร้างฐานข้อมูลเชิงสัมพันธ์ด้วย PostgreSQL ให้เป็นแหล่งข้อมูลกลางของโรงแรมกรณีศึกษา ซึ่งครอบคลุมงานตั้งแต่การจองห้องพัก การชำระเงิน ไปจนถึงงานแม่บ้านและรายงานสำหรับผู้บริหาร เป้าหมายคือแก้ปัญหาการจองห้องซ้อน ยอดเงินคลาดเคลื่อน รวมถึงข้อมูลที่กระจายอยู่หลายแหล่ง

เนื้อหาในรายงานฉบับนี้เรียบเรียงตามขั้นตอนการออกแบบฐานข้อมูลที่ได้ศึกษาในรายวิชา โดยเริ่มจากการวิเคราะห์ปัญหาและความต้องการของระบบ ต่อด้วยการออกแบบเชิงแนวคิดด้วย ER Diagram การออกแบบเชิงตรรกะ ไปจนถึงการจัดโครงสร้างข้อมูลให้อยู่ในรูปแบบบรรทัดฐาน (normalization) จากนั้นจึงสร้างฐานข้อมูลจริงด้วยกลไก 4 ประเภท ได้แก่ ข้อกำหนดบังคับ (constraint) มุมมองข้อมูล (view) ฟังก์ชัน (function) และคำสั่งกระตุ้น (trigger) ส่วนท้ายของรายงานนำเสนอผลการทดสอบกับข้อมูลจำนวนมาก รวมถึงข้อจำกัดที่ยังคงมีอยู่ของระบบ

คณะผู้จัดทำขอขอบพระคุณ รองศาสตราจารย์ ดร. อานนท์ รุ่งสว่าง อาจารย์ผู้สอน ที่กรุณาให้ความรู้และคำแนะนำตลอดภาคการศึกษา คณะผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจการออกแบบฐานข้อมูลเพื่อนำไปใช้กับระบบงานจริง

คณะผู้จัดทำ

สารบัญ

หน้า

บทที่ 1 บทนำ 1
1.1 ที่มาและความสำคัญของปัญหา 1
1.2 วัตถุประสงค์ 1
1.3 ขอบเขตของระบบ 2
1.4 ผู้ใช้งานระบบ 3
1.5 เครื่องมือที่ใช้ 3
บทที่ 2 ความต้องการของระบบ 4
2.1 ความต้องการเชิงหน้าที่ 4
2.2 กฎทางธุรกิจที่สำคัญ 4
2.3 ความต้องการที่ไม่ใช่เชิงหน้าที่ 6
บทที่ 3 การออกแบบเชิงแนวคิด (ER Model) 7
3.1 Entity และการจัดกลุ่ม 7
3.2 ความสัมพันธ์และ Cardinality 8
3.3 ความสัมพันธ์แบบ M:N 9
3.4 ER Diagram 9
บทที่ 4 การออกแบบเชิงตรรกะและ Normalization 10
4.1 Relational Schema 10
4.2 Primary Key และ Candidate Key 11
4.3 Functional Dependencies 11
4.4 กระบวนการทำให้เป็นบรรทัดฐาน (Normalization) 12
4.5 ข้อมูลที่เก็บซ้ำโดยตั้งใจ 13
บทที่ 5 การสร้างฐานข้อมูล (Physical Implementation) 14
5.1 สคริปต์ฐานข้อมูล 14
5.2 ชนิดข้อมูล 14
5.3 Constraint 15
5.4 Index 15
5.5 View 16

5.6 Function 16
5.7 Trigger 17
5.8 สิทธิ์และความปลอดภัยของฐานข้อมูล 18
บทที่ 6 การสอบถามข้อมูลทางธุรกิจ (Business Queries) 19
6.1 ข้อมูลที่ใช้ทดสอบ 19
6.2 รายการ Query 19
6.3 ตัวอย่าง Query และผลลัพธ์ 20
บทที่ 7 การทดสอบฐานข้อมูล 25
7.1 ทดสอบ trigger กันการจองซ้อน (Q16) 25
7.2 ทดสอบ trigger เมื่อเช็กเอาต์ (Q22) 25
7.3 สรุปผลการทดสอบ 26
บทที่ 8 การนำฐานข้อมูลไปใช้งาน 27
บทที่ 9 สรุปและข้อเสนอแนะ 29
9.1 สรุปผล 29
9.2 ข้อจำกัด 29
9.3 ข้อเสนอแนะ 29
9.4 สิ่งที่ได้เรียนรู้ 30

สารบัญตาราง

หน้า

ตารางที่ 1.1 โมดูลของระบบ 2
ตารางที่ 1.2 บทบาทผู้ใช้ 6 บทบาท (ค่าของ app_user.role) 3
ตารางที่ 1.3 เครื่องมือที่ใช้ในโครงงาน 3
ตารางที่ 2.1 สรุปความต้องการเชิงหน้าที่ 4
ตารางที่ 2.2 กฎทางธุรกิจที่สำคัญ 5
ตารางที่ 3.1 Entity 19 ตาราง แบ่งเป็น 6 กลุ่ม 7
ตารางที่ 3.2 ความสัมพันธ์หลักระหว่าง entity 8
ตารางที่ 4.1 Functional dependencies หลัก 11
ตารางที่ 5.1 สคริปต์ฐานข้อมูล 14
ตารางที่ 5.2 สรุป constraint 15
ตารางที่ 5.3 View 7 ตัว 16
ตารางที่ 5.4 Function 3 ตัว 16
ตารางที่ 5.5 ผลของ fn_quote_room(5, วันที่) 17
ตารางที่ 5.6 Trigger 6 ตัว 17
ตารางที่ 6.1 Business queries ใน queries.sql 19
ตารางที่ 6.2 ผลลัพธ์ของ Q1 21
ตารางที่ 6.3 ผลลัพธ์ของ Q4 21
ตารางที่ 6.4 ผลลัพธ์ของ Q7 22
ตารางที่ 6.5 ผลลัพธ์ของ Q11 22
ตารางที่ 6.6 ผลลัพธ์ของ Q13 23
ตารางที่ 6.7 ผลลัพธ์ของ Q19 24
ตารางที่ 7.1 ผลการทดสอบ Q22 25
ตารางที่ 7.2 สรุปผลการทดสอบ 26

สารบัญภาพ

หน้า

ภาพที่ 3.1 ER Diagram ของระบบ 19 ตาราง 25 ความสัมพันธ์ (สร้างด้วย dbdiagram.io)..... 9
ภาพที่ 8.1 สถาปัตยกรรมของระบบ..... 27
ภาพที่ 8.2 หน้าค้นหาห้องว่าง ใช้ fn_available_rooms และ fn_quote_stay..... 27
ภาพที่ 8.3 หน้าภาพรวมผู้บริหาร ใช้ v_room_type_revenue..... 28
ภาพที่ 8.4 ใบกำกับภาษีจากตาราง invoice และ invoice_line..... 28

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของปัญหา

โรงแรมกรณีศึกษาเป็นโรงแรมขนาดกลางในกรุงเทพมหานคร ใช้ชื่อสมมติ Tigger Oriental มีห้องพัก 155 ห้อง 9 ชั้น แบ่งเป็น 5 ประเภทห้อง เดิมบันทึกการจองด้วยสมุดและตารางคำนวณข้อมูลของแต่ละฝ่ายจึงแยกกันอยู่ ทำให้เกิดปัญหาดังนี้

1. ห้องเดียวกันถูกจองซ้อนในช่วงวันเดียวกัน
2. ค้นหาห้องว่างตามช่วงวันได้ช้าและไม่แม่นยำ
3. ยอดเงินคำนวณด้วยมือ คลาดเคลื่อนเมื่อชำระหลายงวดหรือมีการคืนเงิน
4. ราคาห้องเปลี่ยนตามฤดูกาลและวันในสัปดาห์ แต่ไม่มีหลักฐานว่าลูกค้าได้ราคาใด
5. งานทำความสะอาดหลังเช็กเอาต์และงานแจ้งซ่อมตกหล่น
6. สรุปรายได้ได้ช้า และตรวจสอบย้อนหลังไม่ได้ว่าใครทำรายการใด

กลุ่มจึงออกแบบและสร้างฐานข้อมูลเชิงสัมพันธ์ด้วย PostgreSQL 16 เพื่อเป็นแหล่งข้อมูลกลางของทุกฝ่าย โดยให้ฐานข้อมูลบังคับกฎสำคัญด้วย constraint, view, function และ trigger จากนั้นจึงทดลองใช้งานจริงผ่านเว็บแอปพลิเคชันที่เชื่อมต่อกับฐานข้อมูลเดียวกัน

1.2 วัตถุประสงค์

1. เพื่อออกแบบฐานข้อมูลตั้งแต่ ER Diagram, Relational Schema จนถึง Normalization ในรูปแบบบรรทัดฐานที่สาม (3NF)
2. เพื่อสร้างฐานข้อมูลจริงด้วยภาษา SQL พร้อม constraint, view, function และ trigger ที่บังคับกฎทางธุรกิจ
3. เพื่อเขียน business query ที่ตอบคำถามของผู้ใช้แต่ละฝ่าย เช่น ห้องว่าง รายได้ และอัตราการเข้าพัก
4. เพื่อทดสอบการป้องกันการจองซ้อนและความถูกต้องของยอดเงินบนข้อมูลจำนวนมาก

1.3 ขอบเขตของระบบ

ตารางที่ 1.1 โมดูลของระบบ

โมดูล	ความสามารถ	ตารางหลัก
M1 ข้อมูลหลัก	ประเภทห้อง ห้องพัก บริการเสริม ลูกค้า พนักงาน	`room_type`, `room`, `service`, `customer`, `staff`
M2 ห้องว่างและราคา	ค้นหาห้องว่าง เสนอราคารายคืน แผนราคา	`rate_plan` และฟังก์ชัน `fn_available_rooms`, `fn_quote_room`, `fn_quote_stay`
M3 การจอง	จองออนไลน์และจองที่เคาน์เตอร์ หลายห้องต่อการจอง บริการเสริม เลื่อนวัน ยกเลิก เช็กอิน เช็กเอาต์ ไม่มาเข้าพัก	`booking`, `booking_room`, `booking_service`
M4 การเงิน	ชำระหลายงวด คืนเงิน ใบแจ้งหนี้และใบกำกับภาษี	`payment`, `invoice`, `invoice_line`
M5 ลูกค้าสัมพันธ์	ทะเบียนลูกค้า ประวัติการเข้าพัก มูลค่าลูกค้าตลอดอายุ รีวิว	`customer`, `review`
M6 ปฏิบัติการ	งานแม่บ้าน งานแจ้งซ่อม สถานะห้องและประวัติ ผู้เข้าพักปัจจุบัน	`housekeeping_task`, `maintenance_ticket`, `room_status_log`
M7 รายงาน	รายได้ตามประเภทห้อง อัตราการเข้าพัก รายได้รายวัน คาดการณ์ ปฏิทินราคา	`view v_room_type_revenue`, `v_daily_revenue` และตารางธุรกรรม
M8 ความปลอดภัยและการตรวจสอบ	บัญชีผู้ใช้ สิทธิ์ตามบทบาท การล็อกบัญชี ประวัติการเข้าสู่ระบบ ประวัติการทำงาน	`app_user`, `auth_event`, `audit_log`

สิ่งที่อยู่นอกขอบเขต ได้แก่ การตัดเงินผ่านช่องทางชำระเงินจริง การส่งอีเมลยืนยัน การเปลี่ยนห้องหลังจอง การรับจองเกินจำนวนห้อง และการบริหารหลายสาขา

1.4 ผู้ใช้งานระบบ

ตารางที่ 1.2 บทบาทผู้ใช้ 6 บทบาท (ค่าของ app_user.role)

บทบาท	ผู้ใช้	หน้าที่หลัก
`customer`	ลูกค้า	ค้นหาห้องว่าง จอง ชำระเงิน ยกเลิก และรีวิวหลังเข้าพัก
`receptionist`	พนักงานต้อนรับ	จองแทนลูกค้า เช็กอิน เช็กเอาต์ รับชำระเงิน ออกใบแจ้งหนี้
`housekeeping`	พนักงานแม่บ้าน	จัดการงานทำความสะอาดและงานแจ้งซ่อม
`accounting`	พนักงานบัญชี	ดูใบแจ้งหนี้และรายได้รายวัน
`manager`	ผู้จัดการ	รายงานผู้บริหาร แผนราคา ข้อมูลห้อง ประวัติการทำงาน
`admin`	ผู้ดูแลระบบ	จัดการบัญชีผู้ใช้และสิทธิ์

1.5 เครื่องมือที่ใช้

ตารางที่ 1.3 เครื่องมือที่ใช้ในโครงงาน

ส่วน	เครื่องมือ
ฐานข้อมูล	PostgreSQL 16 (ส่วนขยาย pgcrypto สำหรับสร้างค่าแฮชรหัสผ่านแบบ bcrypt)
ออกแบบ	ER Diagram สร้างด้วย dbdiagram.io
ทดลองใช้งาน	เว็บแอปพลิเคชัน (FastAPI + React) และ Docker Compose

บทที่ 2 ความต้องการของระบบ

จากการวิเคราะห์ความต้องการของโรงแรม ได้ความต้องการเชิงหน้าที่ 46 ข้อ กฎทางธุรกิจ 30 ข้อ รวมถึงความต้องการที่ไม่ใช่เชิงหน้าที่ 11 ข้อ บทนี้สรุปเฉพาะส่วนที่เกี่ยวข้องกับการออกแบบฐานข้อมูล

2.1 ความต้องการเชิงหน้าที่

ตารางที่ 2.1 สรุปความต้องการเชิงหน้าที่

กลุ่ม	ความต้องการหลัก
ข้อมูลหลัก	ประเภทห้อง ห้องพัก บริการเสริม ลูกค้า พนักงาน
ห้องว่างและราคา	ค้นหาห้องว่างตามช่วงวัน เสนอราคารายคืนตามแผนราคา
การจอง	จองได้หลายห้อง เลื่อนวัน ยกเลิก เช็กอิน เช็กเอาต์
การเงิน	ชำระหลายงวด คืนเงิน ออกใบแจ้งหนี้และใบกำกับภาษี
ลูกค้าสัมพันธ์	ทะเบียนลูกค้า รีวิวหลังเข้าพัก
ปฏิบัติการ	งานแม่บ้านอัตโนมัติ งานแจ้งซ่อม ประวัติสถานะห้อง
รายงาน	รายได้ตามประเภทห้อง อัตราการเข้าพัก รายได้รายวัน
ความปลอดภัยและการตรวจสอบ	บัญชีผู้ใช้ สิทธิ์ตามบทบาท ประวัติการเข้าสู่ระบบและการทำงาน

2.2 กฎทางธุรกิจที่สำคัญ

กฎทางธุรกิจต่อไปนี้เป็นกฎที่ส่งผลต่อการออกแบบฐานข้อมูลโดยตรง คอลัมน์ขวาแสดงกลไกในฐานข้อมูลที่ใช้บังคับกฎนั้น (คัดมา 13 ข้อจากทั้งหมด 30 ข้อ)

ตารางที่ 2.2 กฎทางธุรกิจที่สำคัญ

รหัส	กฎทางธุรกิจ	กลไกที่บังคับใช้
BR-3	การจองหนึ่งรายการมีได้หลายห้อง แต่ห้องเดียวกันห้ามซ้ำในการจองเดียว	UNIQUE `(booking_id, room_id)`
BR-4	วันเช็กเอาต์ต้องหลังวันเช็กอิน	CHECK `chk_booking_dates`
BR-5	ห้องหนึ่งห้ามถูกจองซ้อนช่วงวันกับการจองอื่นที่ยังมีผล	trigger 2 ตัว
BR-6	ราคาห้องและบริการต้องล็อกไว้ ณ วันจอง ไม่เปลี่ยนตามราคาปัจจุบัน	คอลัมน์ราคาในตารางเชื่อม
BR-8	สถานะการจองมี 5 ค่า และเปลี่ยนตามลำดับที่กำหนด กรณีลูกค้าไม่มาเข้าพัก (no-show) บันทึกเป็น cancelled	CHECK (ค่า) + แอป (ลำดับ)
BR-9	ยอดเงินคำนวณจากข้อมูลต้นทางเสมอ ไม่เก็บยอดรวมซ้ำ (ยกเว้นยอดในใบแจ้งหนี้ ดูหัวข้อ 4.5)	view `v_booking_bill`
BR-11	รีวิวได้ครั้งเดียวต่อการจอง คะแนน 1-5	UNIQUE + CHECK
BR-17	ราคาต่อคืนมาจากแผนราคาที่ priority สูงสุด ถ้าไม่มีใช้ราคาตั้งต้น	function `fn_quote_room`
BR-18	การจองหนึ่งรายการมีใบแจ้งหนี้ที่ใช้งานอยู่ได้ครั้งละ 1 ใบ	partial unique index
BR-20	เช็กเอาต์แล้วต้องสร้างงานทำความสะอาดให้ทุกห้อง	trigger `au_booking_checkout_housekeeping`
BR-24	ห้ามลบข้อมูลหลักที่ธุรกรรมอ้างอิงอยู่ เช่น ลูกค้าที่มีการจอง	FOREIGN KEY ON DELETE RESTRICT
BR-25	รหัสผ่านเก็บเป็นค่าแฮช bcrypt และบัญชีต้องผูกกับลูกค้าหรือพนักงานตามบทบาท	CHECK `chk_user_identity`
BR-27	การสร้างการจองและการเปลี่ยนสถานะการจองทุกครั้งต้องบันทึกผู้กระทำและเวลา	trigger `aiu_booking_audit`

2.3 ความต้องการที่ไม่ใช่เชิงหน้าที่

ความต้องการที่ไม่ใช่เชิงหน้าที่ที่เกี่ยวข้องกับการออกแบบฐานข้อมูลโดยตรงมี 4 ข้อ ดังนี้

- ความปลอดภัย: เก็บรหัสผ่านเป็นค่าแฮช bcrypt และแอปเชื่อมต่อฐานข้อมูลด้วยสิทธิ์เท่าที่จำเป็น
- ความถูกต้อง: กฎที่ฐานข้อมูลบังคับได้ต้องบังคับด้วย constraint หรือ trigger และทุกคำขอทำงานใน transaction เดียว
- ประสิทธิภาพ: มี index บนคีย์นอกและคอลัมน์ที่ค้นหาบ่อย
- การตรวจสอบย้อนหลัง: บันทึกผู้กระทำและเวลาของการเปลี่ยนสถานะการจองทุกครั้ง

บทที่ 3 การออกแบบเชิงแนวคิด (ER Model)

จากกระบวนการทำงานของโรงแรม ได้ entity ทั้งหมด 19 ตัว มีความสัมพันธ์ (foreign key) 25 เส้น จัดเป็น 6 กลุ่มตามหน้าที่

3.1 Entity และการจัดกลุ่ม

ตารางที่ 3.1 Entity 19 ตาราง แบ่งเป็น 6 กลุ่ม

กลุ่ม	Entity (ตาราง)	ความหมาย
People	`customer`, `staff`	บุคคล: ลูกค้าและพนักงาน
Auth	`app_user`, `auth_event`, `audit_log`	บัญชีเข้าสู่ระบบ ประวัติการเข้าสู่ระบบ และประวัติการทำงานของระบบ
Catalog	`room_type`, `room`, `service`, `rate_plan`	ข้อมูลหลักที่ใช้ขาย: ประเภทห้อง ห้อง บริการเสริม แผนราคา
Booking	`booking`, `booking_room`, `booking_service`, `review`	ธุรกรรมการจอง ตารางเชื่อม M:N และรีวิว
Billing	`payment`, `invoice`, `invoice_line`	เงินที่รับจริงและเอกสารเรียกเก็บเงิน
Operations	`room_status_log`, `housekeeping_task`, `maintenance_ticket`	งานปฏิบัติการของห้องพัก

3.2 ความสัมพันธ์และ Cardinality

ตารางที่ 3.2 ความสัมพันธ์หลักระหว่าง entity

ความสัมพันธ์	Cardinality	ความหมาย
room_type - room	1 : N	ห้องแต่ละห้องมีประเภทเดียว ประเภทหนึ่งมีได้หลายห้อง
room_type - rate_plan	1 : N	ประเภทห้องมีแผนราคาได้หลายแผน (ฤดูกาล วันหยุด โปรโมชัน)
customer - booking	1 : N	ลูกค้าหนึ่งคนจองได้หลายครั้ง การจองหนึ่งรายการเป็นของลูกค้าคนเดียว
staff - booking	1 : N	พนักงานผู้รับผิดชอบการจอง (จองออนไลน์ผูกกับพนักงาน Online Reservations)
booking - room	M : N	การจองมีหลายห้อง และห้องอยู่ในหลายการจองต่างช่วงเวลา
booking - service	M : N	การจองมีบริการเสริมได้หลายรายการ
booking - payment	1 : N	ชำระได้หลายงวด การคืนเงินเป็นแถวใหม่
booking - review	1 : 0..1	รีวิวได้ไม่เกินหนึ่งครั้งต่อการจอง
booking - invoice	1 : N (ใช้งานได้ 1)	แก้ใบแจ้งหนี้ด้วยการ void แล้วออกใหม่ ใบที่ void เก็บเป็นหลักฐาน
invoice - invoice_line	1 : N	บรรทัดค่าห้อง ค่าบริการ ส่วนลด
room - housekeeping_task	1 : N	งานทำความสะอาดของห้อง
room - maintenance_ticket	0..1 : N	งานซ่อมของห้อง (NULL = พื้นที่ส่วนกลาง)
customer - app_user	1 : 0..1	ลูกค้ามีบัญชีเข้าสู่ระบบได้ไม่เกินหนึ่งบัญชี
staff - app_user	1 : 0..1	พนักงานมีบัญชีเข้าสู่ระบบได้ไม่เกินหนึ่งบัญชี

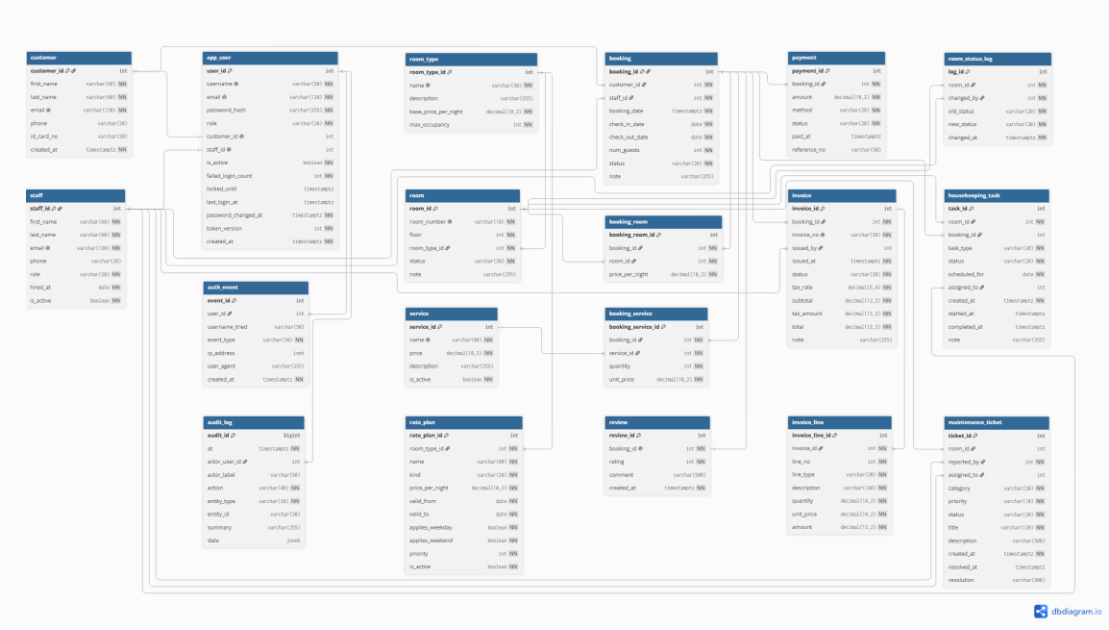
3.3 ความสัมพันธ์แบบ M:N

ความสัมพันธ์แบบ many-to-many มี 2 คู่ แก้ด้วยตารางเชื่อม (junction table) ซึ่งมี attribute ของความสัมพันธ์เอง

- `booking_room` เชื่อมการจองกับห้อง เก็บราคาต่อคืนที่ล็อกไว้ และมี UNIQUE `(booking_id, room_id)`
- `booking_service` เชื่อมการจองกับบริการเสริม เก็บจำนวนและราคาต่อหน่วย และมี UNIQUE `(booking_id, service_id)`

3.4 ER Diagram

ภาพที่ 3.1 เป็น ER Diagram ของระบบที่สร้างด้วย dbdiagram.io แสดงตารางทั้ง 19 ตาราง ความสัมพันธ์ 25 เส้น รวมถึงชนิดข้อมูลและ NOT NULL (NN) ของทุกคอลัมน์ ส่วน cardinality ของแต่ละความสัมพันธ์สรุปไว้ในตารางที่ 3.2



ภาพที่ 3.1 ER Diagram ของระบบ 19 ตาราง 25 ความสัมพันธ์ (สร้างด้วย dbdiagram.io)

บทที่ 4 การออกแบบเชิงตรรกะและ Normalization

4.1 Relational Schema

แปลง ER Diagram เป็น 19 รีเลชัน โดยตัวหนาขีดเส้นใต้คือ Primary Key และตัวเอียงคือ Foreign Key

room_type (**room_type_id**, name, description, base_price_per_night, max_occupancy)

room (**room_id**, room_number, floor, *room_type_id*, status, note)

customer (**customer_id**, first_name, last_name, email, phone, id_card_no, created_at)

staff (**staff_id**, first_name, last_name, email, phone, role, hired_at, is_active)

service (**service_id**, name, price, description, is_active)

app_user (**user_id**, username, email, password_hash, role, *customer_id*, *staff_id*, is_active, failed_login_count, locked_until, last_login_at, password_changed_at, token_version, created_at)

auth_event (**event_id**, *user_id*, username_tried, event_type, ip_address, user_agent, created_at)

booking (**booking_id**, *customer_id*, *staff_id*, booking_date, check_in_date, check_out_date, num_guests, status, note)

booking_room (**booking_room_id**, *booking_id*, *room_id*, price_per_night)

booking_service (**booking_service_id**, *booking_id*, *service_id*, quantity, unit_price)

payment (**payment_id**, *booking_id*, amount, method, status, paid_at, reference_no)

review (**review_id**, *booking_id*, rating, comment, created_at)

room_status_log (**log_id**, *room_id*, *changed_by*, old_status, new_status, changed_at)

rate_plan (**rate_plan_id**, *room_type_id*, name, kind, price_per_night, valid_from, valid_to, applies_weekday, applies_weekend, priority, is_active)

invoice (**invoice_id**, *booking_id*, invoice_no, *issued_by*, issued_at, status, tax_rate, subtotal, tax_amount, total, note)

invoice_line (**invoice_line_id**, *invoice_id*, line_no, line_type, description, quantity, unit_price, amount)

housekeeping_task (**task_id**, *room_id*, *booking_id*, task_type, status, scheduled_for, *assigned_to*, created_at, started_at, completed_at, note)

maintenance_ticket (**ticket_id**, *room_id*, *reported_by*, *assigned_to*, category, priority, status, title, description, created_at, resolved_at, resolution)

audit_log (**audit_id**, at, *actor_user_id*, actor_label, action, entity_type, entity_id, summary, data)

4.2 Primary Key และ Candidate Key

ทุกตารางใช้ surrogate key แบบ `SERIAL` เป็น Primary Key เพราะค่าไม่เปลี่ยนตามข้อมูลธุรกิจ ยกเว้น `audit_log` ใช้ `BIGSERIAL` เพราะจำนวนแถวเพิ่มขึ้นเร็วกว่าตารางอื่นมาก ส่วนคีย์ธรรมชาติ เช่น เลขห้อง อีเมลลูกค้า และเลขที่ใบแจ้งหนี้ ยังกำหนด UNIQUE constraint ไว้ จึงทำหน้าที่เป็น candidate key

4.3 Functional Dependencies

ตารางที่ 4.1 Functional dependencies หลัก

#	Functional Dependency
FD1	booking_no ⟶ booking_date, customer_email, staff_email, check_in, check_out, num_guests, status
FD2	customer_email ⟶ customer_name, customer_phone
FD3	staff_email ⟶ staff_name, role
FD4	room_no ⟶ floor, room_type
FD5	room_type ⟶ base_price, max_occupancy
FD6	(booking_no, room_no) ⟶ price_per_night
FD7	service_name ⟶ service_price
FD8	(booking_no, service_name) ⟶ quantity, unit_price

4.4 กระบวนการทำให้เป็นบรรทัดฐาน (Normalization)

ข้อมูลตั้งต้นคือ "ใบจองห้องพัก" หนึ่งแถวต่อการจอง ซึ่งมีกลุ่มข้อมูลซ้ำของห้อง บริการ และการชำระเงิน (UNF)

```
BOOKING_SHEET ( booking_no, booking_date, customer_name, customer_email,
                customer_phone, staff_name, staff_email, check_in, check_out,
                num_guests, status,
                { room_no, floor, room_type, base_price, max_occupancy,
                  price_per_night },
                { service_name, service_price, quantity, unit_price },
                { payment_no, pay_amount, pay_method, pay_status, paid_at } )
```

1NF แยกกลุ่มซ้ำออกให้ทุกค่าเป็นค่าเดี่ยว ข้อมูลห้องกลายเป็นหนึ่งแถวต่อห้องของการจอง ได้คีย์ประกอบ (booking_no, room_no) ส่วนบริการและการชำระเงินแยกเป็นรีเลชันของตัวเอง

2NF ต้องไม่มี partial dependency ข้อมูลการจองขึ้นกับ booking_no อย่างเดียว (FD1) และข้อมูลห้องขึ้นกับ room_no อย่างเดียว (FD4) จึงแยกเป็น BOOKING, ROOM และ BOOKING_ROOM ซึ่งเก็บเฉพาะ price_per_night ที่ขึ้นกับคีย์ทั้งชุด (FD6) ส่วน service_price ขึ้นกับ service_name อย่างเดียว (FD7) ไม่ได้ขึ้นกับคีย์ทั้งชุด (booking_no, service_name) จึงแยก SERVICE ออกจาก BOOKING_SERVICE

3NF ต้องไม่มี transitive dependency ยังเหลือ booking_no ⟶ customer_email ⟶ customer_name (FD2), booking_no ⟶ staff_email ⟶ staff_name (FD3) และ room_no ⟶ room_type ⟶ base_price (FD5) จึงแยกเป็น CUSTOMER, STAFF และ ROOM_TYPE

```
CUSTOMER        ( customer_email, customer_name, customer_phone )
STAFF           ( staff_email, staff_name, role )
ROOM_TYPE       ( room_type, base_price, max_occupancy )
ROOM            ( room_no, floor, room_type )
BOOKING         ( booking_no, booking_date, customer_email, staff_email,
                  check_in, check_out, num_guests, status )
BOOKING_ROOM    ( booking_no, room_no, price_per_night )
SERVICE         ( service_name, service_price )
BOOKING_SERVICE ( booking_no, service_name, quantity, unit_price )
PAYMENT         ( payment_no, booking_no, pay_amount, pay_method, pay_status,
                  paid_at )
```

สุดท้ายแทนคีย์ธรรมชาติด้วย surrogate key ได้ 9 ตารางแกนหลักของ `schema.sql` ตารางที่เพิ่มภายหลังออกแบบตามหลักเดียวกัน ทุกตารางจึงอยู่ในรูปแบบ 3NF ยกเว้นคอลัมน์ที่ตั้งใจเก็บซ้ำตามหัวข้อ 4.5

4.5 ข้อมูลที่เก็บซ้ำโดยตั้งใจ

ค่าที่คำนวณได้ เช่น ยอดรวม ยอดคงค้าง และจำนวนคืน ไม่ถูกเก็บในตาราง แต่คำนวณผ่าน view ยกเว้นข้อมูลต่อไปนี้ที่ตั้งใจเก็บ

- `booking_room.price_per_night` และ `booking_service.unit_price` คือราคา ณ วันจอง ซึ่งเป็นข้อเท็จจริงคนละเวลากับราคาปัจจุบัน (BR-6)
- ยอดเงินในใบแจ้งหนี้ (`invoice.subtotal`, `invoice.tax_amount`, `invoice.total` และ `invoice_line.amount`) ต้องคงค่า ณ วันออกเอกสาร แม้ total จะคำนวณได้จาก subtotal + tax_amount ก็ต้องเก็บไว้ตามที่พิมพ์ในเอกสารภาษี
- `staff.role` กับ `app_user.role` มีค่าเดียวกัน จึงใช้ trigger 2 ตัวทำให้ตรงกันเสมอ
- `app_user.email` เก็บแยกจาก `customer.email` และ `staff.email` เพราะใช้สำหรับเข้าสู่ระบบ ระบบคัดลอกอีเมลเดียวกันตอนสร้างบัญชี แต่ฐานข้อมูลไม่ได้บังคับให้ตรงกันหลังจากนั้น

บทที่ 5 การสร้างฐานข้อมูล (Physical Implementation)

5.1 สคริปต์ฐานข้อมูล

ตารางที่ 5.1 สคริปต์ฐานข้อมูล

ลำดับ	ไฟล์	เนื้อหา
1	`schema.sql`	สร้าง 19 ตาราง พร้อม constraint, index, view, function และ trigger
2	`roles.sql`	สร้าง role `app_rw` (อ่าน/เขียนข้อมูล) และ `app_readonly` (อ่านอย่างเดียว)
3	`seed.sql`	ข้อมูลตัวอย่าง 155 ห้อง 1,000 ลูกค้า 2,484 การจอง
4	`queries.sql`	Business queries Q1-Q22

5.2 ชนิดข้อมูล

- Primary key เป็น `SERIAL` (และ `BIGSERIAL` สำหรับ `audit_log`)
- จำนวนเงินใช้ `NUMERIC(10,2)` และ `NUMERIC(12,2)` สำหรับยอดในใบแจ้งหนี้ ไม่ใช้ทศนิยมลอยตัว เพื่อไม่ให้ยอดเงินคลาดเคลื่อน
- วันเข้าพักเป็น `DATE` ส่วนเวลาที่เกิดเหตุการณ์เป็น `TIMESTAMPTZ`
- ค่าที่มีชุดจำกัด เช่น สถานะ ใช้ `VARCHAR` คู่กับ `CHECK ... IN (...)`

5.3 Constraint

ตารางที่ 5.2 สรุป constraint

ชนิด	จำนวน	ตัวอย่าง
PRIMARY KEY	19	ทุกตาราง
FOREIGN KEY	25	RESTRICT 7 เส้น, CASCADE 11 เส้น, SET NULL 7 เส้น
UNIQUE	14	`customer.email`, `room.room_number`, `uq_booking_room`
CHECK	40	`check_out_date > check_in_date`, `rating BETWEEN 1 AND 5`, ชุดค่าสถานะ
รวม	**98**	ไม่รวม NOT NULL และ partial unique index

```
CONSTRAINT chk_booking_dates CHECK (check_out_date > check_in_date)  -- BR-4
CONSTRAINT uq_booking_room UNIQUE (booking_id, room_id)  -- BR-3
CREATE UNIQUE INDEX uq_invoice_active_booking ON invoice (booking_id)
       WHERE status <> 'void';  -- BR-18
```

5.4 Index

นอกจาก index ที่ PostgreSQL สร้างให้อัตโนมัติจาก PRIMARY KEY กับ UNIQUE แล้ว กลุ่มได้สร้าง index เพิ่มอีก 28 ตัว ครอบคลุมคอลัมน์ foreign key ทุกคอลัมน์ รวมถึงคอลัมน์ที่ค้นหาบ่อย ตัวอย่างเช่น

- `idx_booking_dates (check_in_date, check_out_date)` ใช้ค้นหาห้องว่างตามช่วงวัน
- `idx_booking_status (status)` ใช้กรองการจองตามสถานะ
- `idx_booking_room_room (room_id)` ใช้หาการจองของแต่ละห้อง

5.5 View

ตารางที่ 5.3 View 7 ตัว

View	ใช้ทำอะไร
`v_booking_bill`	ยอดเงินของแต่ละการจอง (BR-9)
`v_current_guests`	ผู้เข้าพักที่เช็กอินอยู่ขณะนี้
`v_room_type_revenue`	รายได้ตามประเภทห้อง (ไม่นับการจองที่ยกเลิก)
`v_housekeeping_board`	กระดานงานแม่บ้าน
`v_maintenance_open`	งานซ่อมที่ยังไม่ปิด
`v_daily_revenue`	รายได้ค่าห้องรายวันตามคืนที่เข้าพักจริง
`v_invoice_summary`	ใบแจ้งหนี้พร้อมยอดชำระและยอดคงเหลือ

`v_booking_bill` คำนวณยอดเงินของทุกการจองจากข้อมูลต้นทาง จึงไม่ต้องเก็บยอดรวมไว้ในตาราง **booking**

```
grand_total = ค่าห้องทุกห้อง × จำนวนคืน + ค่าบริการเสริม
balance_due = grand_total - ยอดที่ชำระแล้ว + ยอดที่คืนเงินแล้ว  (การจองที่ยกเลิกเป็น 0)
```

5.6 Function

ตารางที่ 5.4 Function 3 ตัว

Function	หน้าที่
`fn_available_rooms(check_in, check_out)`	คืนรายการห้องที่ว่างและพร้อมขายในช่วงวันที่กำหนด
`fn_quote_room(room_type_id, night)`	ราคาของห้องประเภทนั้นในคืนที่กำหนดตามแผนราคา
`fn_quote_stay(room_type_id, check_in, check_out)`	ผลรวมราคาทุกคืนของการเข้าพัก

ตัวอย่างการทำงานของ `fn_quote_room` กับห้อง Oriental Suite (ราคาตั้งต้น 3,800 บาท) ในเดือนธันวาคม 2569

ตารางที่ 5.5 ผลของ fn_quote_room(5, วันที่)

คืนวันที่ (ค.ศ.)	วัน	ราคา (บาท)	เหตุผล
2026-12-18	ศุกร์	3800.00	ไม่มีแผนตรง ใช้ราคาตั้งต้น
2026-12-19	เสาร์	4400.00	แผน Weekend (priority 10)
2026-12-21	จันทร์	5200.00	แผน High season (priority 50)
2026-12-26	เสาร์	5200.00	High season ชนะ Weekend เพราะ priority สูงกว่า

5.7 Trigger

ตารางที่ 5.6 Trigger 6 ตัว

Trigger	ทำงานเมื่อ	หน้าที่
`bi_booking_room_no_overlap`	เพิ่ม/แก้ห้องของการจอง	ปฏิเสธถ้าห้องถูกจองซ้อนช่วงวัน (BR-5)
`bu_booking_no_overlap`	แก้วันที่หรือสถานะการจอง	ตรวจการจองซ้อนอีกครั้งเมื่อเลื่อนวันหรือเปลี่ยนสถานะ (BR-5)
`au_booking_checkout_housekeeping`	การจองเปลี่ยนเป็น checked_out	สร้างงานทำความสะอาด และตั้งห้องที่ยังเป็น available ให้เป็น cleaning (BR-20)
`aiu_booking_audit`	สร้างหรือเปลี่ยนสถานะการจอง	บันทึกประวัติลง `audit_log` (BR-27)
`aiu_app_user_role_sync`	เพิ่มบัญชีหรือแก้บทบาทใน `app_user`	ทำให้ `staff.role` ตรงกัน
`au_staff_role_sync`	แก้บทบาทใน `staff`	ทำให้ `app_user.role` ตรงกัน

กฎห้ามจองซ้อนตรวจด้วย CHECK ไม่ได้ เพราะต้องเทียบกับแถวอื่นและข้ามตาราง จึงใช้ trigger นับการจองอื่นของห้องเดียวกันที่ช่วงวันทับกัน ถ้าพบให้ยกเลิกคำสั่งทันที

และระหว่างตรวจจะล็อกห้องนั้นไว้ชั่วคราว ถ้ามีคนจองห้องเดียวกันพร้อมกัน
คนที่สองต้องรอแล้วถูกปฏิเสธ

```
-- ล็อกต่อห้องจนจบ transaction ถ้าจองห้องเดียวกันพร้อมกัน คนที่สองต้องรอ
PERFORM pg_advisory_xact_lock(5, NEW.room_id);
-- ช่วงวันทับกันเมื่อ: เช็กอินของ A < เช็กเอาต์ของ B  และ  เช็กอินของ B < เช็กเอาต์ของ A
IF (SELECT COUNT(*) FROM booking_room br JOIN booking b USING (booking_id)
    WHERE br.room_id = NEW.room_id
      AND br.booking_id <> NEW.booking_id      -- ไม่นับการจองของตัวเอง
      AND b.status IN ('pending','confirmed','checked_in')
      AND b.check_in_date < v_check_out AND v_check_in < b.check_out_date) > 0 THEN
    RAISE EXCEPTION '... (double booking)';
END IF;
```

5.8 สิทธิ์และความปลอดภัยของฐานข้อมูล

- แอปเชื่อมต่อด้วย role `app_rw` ซึ่งอ่านและเขียนข้อมูลได้ แต่สร้างหรือลบตารางไม่ได้ ส่วนรายงานใช้ `app_readonly`
- รหัสผ่านเก็บเป็นค่าแฮช bcrypt เท่านั้น
- ทุกคำสั่ง SQL ในแอปใช้พารามิเตอร์ ป้องกัน SQL Injection

บทที่ 6 การสอบถามข้อมูลทางธุรกิจ (Business Queries)

6.1 ข้อมูลที่ใช้ทดสอบ

business query ทุกข้อรันเมื่อวันที่ 29 กันยายน 2569 บนฐานข้อมูลที่โหลด `seed.sql` แล้วเพิ่มข้อมูลประวัติการดำเนินงานปี 2543-2569 เพื่อทดสอบกับข้อมูลจำนวนมาก ได้แก่ การจอง 342,406 รายการ ลูกค้า 248,832 คน การชำระเงิน 507,624 รายการ และใบแจ้งหนี้ 317,936 ใบ

6.2 รายการ Query

ตารางที่ 6.1 Business queries ใน queries.sql

#	โจทย์	เทคนิค SQL	แถว
Q1	ห้องว่างช่วง 1-5 ต.ค. 2569	INNER JOIN, NOT EXISTS, correlated subquery	7
Q2	ใบแจ้งยอดของทุกการจอง	VIEW `v_booking_bill`	342,406
Q3	ผู้เข้าพักที่เช็กอินอยู่ และคืนที่เหลือ	JOIN 5 ตาราง, GREATEST	94
Q4	รายได้ตามประเภทห้อง	GROUP BY, SUM, COUNT DISTINCT	5
Q5	อัตราการเข้าพักรายเดือน	CTE, CROSS JOIN LATERAL generate_series	321
Q6	ลูกค้าที่ใช้จ่ายสูงสุด 5 อันดับ	aggregate บน VIEW, ORDER BY, LIMIT	5
Q7	จัดอันดับรายได้ของแต่ละห้อง	LEFT JOIN, COALESCE, window RANK()	155
Q8	การจองที่ยังค้างชำระ	VIEW + JOIN + filter	967
Q9	คะแนนรีวิวเฉลี่ยตามประเภทห้อง	subquery DISTINCT, AVG / MIN / MAX	5
Q10	ผลงานพนักงาน: การจองที่ดูแลและยอดขาย	LEFT JOIN, CASE ใน SUM	63
Q11	ลูกค้าที่ไม่เคยจอง	anti-join (LEFT JOIN … IS NULL)	2
Q12	บริการเสริมยอดนิยมและส่วนแบ่งรายได้	SUM + window SUM() OVER ()	6
Q13	ตรวจกฎห้ามจองซ้อน (ต้องได้ 0 แถว)	self-join, เงื่อนไขช่วงวันซ้อน	0
Q14	[Security] ล็อกอินล้มเหลวซ้ำใน 7 วัน	GROUP BY, HAVING, scalar subquery	0

#	โจทย์	เทคนิค SQL	แถว
Q15	[Security] บัญชีผู้ใช้ ตัวตน และสถานะล็อก	LEFT JOIN 2 ทาง, COALESCE, CASE	10
Q16	ทดสอบ trigger กันจองซ้อน	CTE INSERT … RETURNING, transaction + ROLLBACK	ทดสอบ
Q17	ราคาต่อคืนตามแผนราคาเทียบราคาตั้งต้น	generate_series, function `fn_quote_room`	5
Q18	แผนราคาที่เปิดใช้งาน	JOIN, CASE, ORDER BY priority	348
Q19	กระทบยอดใบแจ้งหนี้กับบรรทัดรายการ	GROUP BY, VIEW, นิพจน์ boolean	317,936
Q20	ภาระงานแม่บ้านรายคน	LEFT JOIN, COUNT … FILTER	37
Q21	ห้องที่มีงานซ่อมค้าง และคืนที่เคยขายได้	LEFT JOIN, CASE ใน ORDER BY	3
Q22	ทดสอบ trigger เช็กเอาต์ + audit	UPDATE ใน transaction, SET LOCAL, ROLLBACK	ทดสอบ

6.3 ตัวอย่าง Query และผลลัพธ์

Q1 ห้องว่างช่วง 1-5 ต.ค. 2569

เทคนิค: INNER JOIN, NOT EXISTS, correlated subquery

```
SELECT r.room_number, rt.name AS room_type, rt.base_price_per_night, rt.max_occupancy
FROM room r
JOIN room_type rt ON rt.room_type_id = r.room_type_id
WHERE r.status = 'available'
  AND NOT EXISTS (
      SELECT 1
      FROM booking_room br
      JOIN booking b ON b.booking_id = br.booking_id
      WHERE br.room_id = r.room_id
        AND b.status IN ('pending','confirmed','checked_in')
        AND b.check_in_date < DATE '2026-10-05'
        AND DATE '2026-10-01' < b.check_out_date
  )
ORDER BY rt.base_price_per_night, r.room_number;
```

ตารางที่ 6.2 ผลลัพธ์ของ Q1

room_number	room_type	base_price_per_night	max_occupancy
104	Parkview Room	1300.00	2
201	Parkview Room	1300.00	2
412	Parkview Room	1300.00	2
510	Parkview Room	1300.00	2
704	Parkview Room	1300.00	2

แสดง 5 แถวแรกจากทั้งหมด 7 แถว

เงื่อนไข `r.status = 'available'` ตั้งใจตัดห้องที่ยังไม่พร้อมขาย ณ วันที่รัน (กำลังทำความสะอาด ซ่อม หรือปิดใช้งาน) ออกก่อน แม้ห้องเหล่านั้นอาจว่างในช่วง 1-5 ต.ค. ผลลัพธ์ 7 ห้อง จึงเป็นห้องที่ว่างตามช่วงวันและพร้อมขายทันที

Q4 รายได้ตามประเภทห้อง

เทคนิค: GROUP BY, SUM, COUNT DISTINCT

```
SELECT rt.name AS room_type,
       COUNT(DISTINCT br.booking_id)                                  AS bookings,
       SUM(bk.check_out_date - bk.check_in_date)                      AS room_nights,
       SUM(br.price_per_night * (bk.check_out_date - bk.check_in_date)) AS revenue
FROM booking_room br
JOIN booking bk   ON bk.booking_id = br.booking_id AND bk.status <> 'cancelled'
JOIN room rm      ON rm.room_id = br.room_id
JOIN room_type rt ON rt.room_type_id = rm.room_type_id
GROUP BY rt.name
ORDER BY revenue DESC;
```

ตารางที่ 6.3 ผลลัพธ์ของ Q4

room_type	bookings	room_nights	revenue
Parkview Room	132466	416534	424642055.56
Premier Skyline Room	69238	220803	318597603.49
Oriental Suite	17978	73527	222741870.29
Cityview Room	86371	272435	196264235.12
Skyline Residence	13410	54089	105774644.05

Q7 จัดอันดับรายได้ของแต่ละห้อง

เทคนิค: LEFT JOIN, COALESCE, window RANK()

```
SELECT rm.room_number,
       rt.name AS room_type,
       COALESCE(SUM(br.price_per_night * (b.check_out_date - b.check_in_date)), 0) AS revenue,
       RANK() OVER (ORDER BY COALESCE(SUM(br.price_per_night *
                    (b.check_out_date - b.check_in_date)), 0) DESC)              AS revenue_rank
FROM room rm
JOIN room_type rt     ON rt.room_type_id = rm.room_type_id
LEFT JOIN booking_room br ON br.room_id = rm.room_id
LEFT JOIN booking b       ON b.booking_id = br.booking_id AND b.status <> 'cancelled'
GROUP BY rm.room_number, rt.name
ORDER BY revenue_rank, rm.room_number;
```

ตารางที่ 6.4 ผลลัพธ์ของ Q7

room_number	room_type	revenue	revenue_rank
815	Oriental Suite	22466770.38	1
916	Oriental Suite	22454539.83	2
915	Oriental Suite	22409879.95	3
304	Oriental Suite	22348569.82	4
816	Oriental Suite	22220650.19	5

แสดง 5 แถวแรกจากทั้งหมด 155 แถว

Q11 ลูกค้าที่ไม่เคยจอง

เทคนิค: anti-join (LEFT JOIN … IS NULL)

```
SELECT c.customer_id, c.first_name || ' ' || c.last_name AS customer_name, c.email
FROM customer c
LEFT JOIN booking b ON b.customer_id = c.customer_id
WHERE b.booking_id IS NULL
ORDER BY c.customer_id;
```

ตารางที่ 6.5 ผลลัพธ์ของ Q11

customer_id	customer_name	email
11	ธนพล แซ่เฮิน	thanaphon.s@example.com
12	Laura Meyer	laura.m@example.com

Q13 ตรวจกฎห้ามจองซ้อน (ต้องได้ 0 แถว)

เทคนิค: self-join, เงื่อนไขช่วงวันซ้อน

```
SELECT br1.room_id,
       b1.booking_id AS booking_a, b1.check_in_date AS in_a, b1.check_out_date AS out_a,
       b2.booking_id AS booking_b, b2.check_in_date AS in_b, b2.check_out_date AS out_b
FROM booking_room br1
JOIN booking_room br2 ON br2.room_id = br1.room_id AND br2.booking_id > br1.booking_id
JOIN booking b1 ON b1.booking_id = br1.booking_id AND b1.status <> 'cancelled'
JOIN booking b2 ON b2.booking_id = br2.booking_id AND b2.status <> 'cancelled'
WHERE b1.check_in_date < b2.check_out_date
  AND b2.check_in_date < b1.check_out_date
ORDER BY br1.room_id;
```

ตารางที่ 6.6 ผลลัพธ์ของ Q13

room_id	booking_a	in_a	out_a	booking_b	in_b	out_b

Q13 ได้ 0 แถว หมายความว่าในข้อมูลการจองทั้งหมดไม่มีห้องใดถูกจองซ้อน

Q19 กระทบยอดใบแจ้งหนี้กับบรรทัดรายการ

เทคนิค: GROUP BY, VIEW, นิพจน์ boolean

```
SELECT i.invoice_no,
       v.customer_name,
       SUM(l.amount)              AS lines_total,
       i.total,
       i.total - SUM(l.amount)    AS diff,
       i.subtotal,
       i.tax_amount,
       i.subtotal + i.tax_amount = i.total AS vat_ok
FROM invoice i
JOIN invoice_line   l ON l.invoice_id = i.invoice_id
JOIN v_invoice_summary v ON v.invoice_id = i.invoice_id
GROUP BY i.invoice_id, i.invoice_no, v.customer_name, i.subtotal, i.tax_amount, i.total
ORDER BY i.invoice_no;
```

ตารางที่ 6.7 ผลลัพธ์ของ Q19

invoice_no	customer_name	lines_total	total	diff	subtotal	tax_amount	vat_ok
INV-2000-00001	Lukas Brown	1730.00	1730.00	0.00	1616.82	113.18	t
INV-2000-00002	Junjie Nguyen	720.00	720.00	0.00	672.90	47.10	t
INV-2000-00003	วรรณา พึ่งทอง	1450.00	1450.00	0.00	1355.14	94.86	t
INV-2000-00004	Yui Sato	1170.00	1170.00	0.00	1093.46	76.54	t

แสดง 4 แถวแรกจากทั้งหมด 317,936 แถว

เมื่อตรวจกับใบแจ้งหนี้ทั้ง 317,936 ใบ ไม่มีใบใดที่ยอดรวมไม่ตรงกับผลรวมบรรทัด และไม่มีใบใดที่ยอดก่อนภาษีบวก VAT ไม่เท่ากับยอดรวม

บทที่ 7 การทดสอบฐานข้อมูล

7.1 ทดสอบ trigger กันการจองซ้อน (Q16)

การจองเลขที่ 6 (สถานะ checked_in) ครองห้อง 203 (room_id = 7) ช่วง 5-9 ก.ย. 2569 ทดลองสร้างการจองใหม่ห้องเดิมช่วง 7-9 ก.ย. 2569 ภายใน transaction

```
BEGIN;
WITH nb AS (
    INSERT INTO booking (customer_id, staff_id, check_in_date, check_out_date,
                         num_guests, status)
    VALUES (5, 2, '2026-09-07', '2026-09-09', 2, 'confirmed')
    RETURNING booking_id
)
INSERT INTO booking_room (booking_id, room_id, price_per_night)
SELECT booking_id, 7, 1800.00 FROM nb;
-- ERROR:  ห้อง room_id=7 ถูกจองในช่วงวันที่ 2026-09-07 ถึง 2026-09-09
--         แล้ว (double booking)
ROLLBACK;
```

ผล: trigger ปฏิเสธคำสั่ง และแถวการจองที่เพิ่มไปก่อนหน้าใน transaction เดียวกันถูกยกเลิกด้วย

7.2 ทดสอบ trigger เมื่อเช็กเอาต์ (Q22)

ตารางที่ 7.1 ผลการทดสอบ Q22

ขั้นตอน	ผลที่ได้
ก่อนเช็กเอาต์	ห้อง 203 สถานะ `available`
เปลี่ยนการจองเลขที่ 6 เป็น `checked_out`	เกิดงานทำความสะอาดใหม่ของห้อง 203 และห้องเปลี่ยนเป็น `cleaning`
ตรวจ `audit_log`	มีบันทึก `booking.status_change` พร้อมชื่อผู้กระทำ `recep01`
ROLLBACK	ข้อมูลกลับเป็นเหมือนเดิม

หมายเหตุ: คอลัมน์ `room.status` เก็บความพร้อมของห้อง (available, cleaning, maintenance, closed) ไม่ได้เก็บว่ามีแขกพักอยู่หรือไม่ ห้อง 203 ที่มีการจองเลขที่ 6 สถานะ checked_in อยู่จึงยังมีสถานะ available ส่วนการมีผู้เข้าพักดูจากการจองที่เป็น checked_in (view `v_current_guests`)

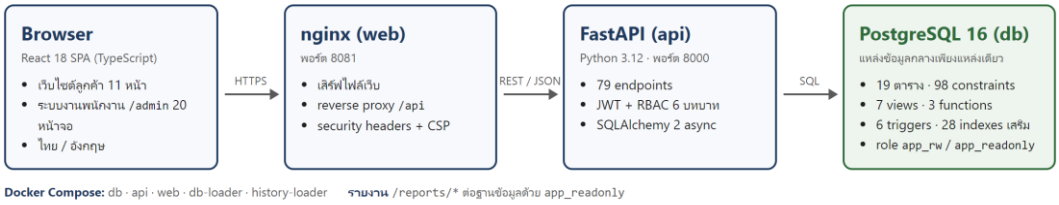
7.3 สรุปผลการทดสอบ

ตารางที่ 7.2 สรุปผลการทดสอบ

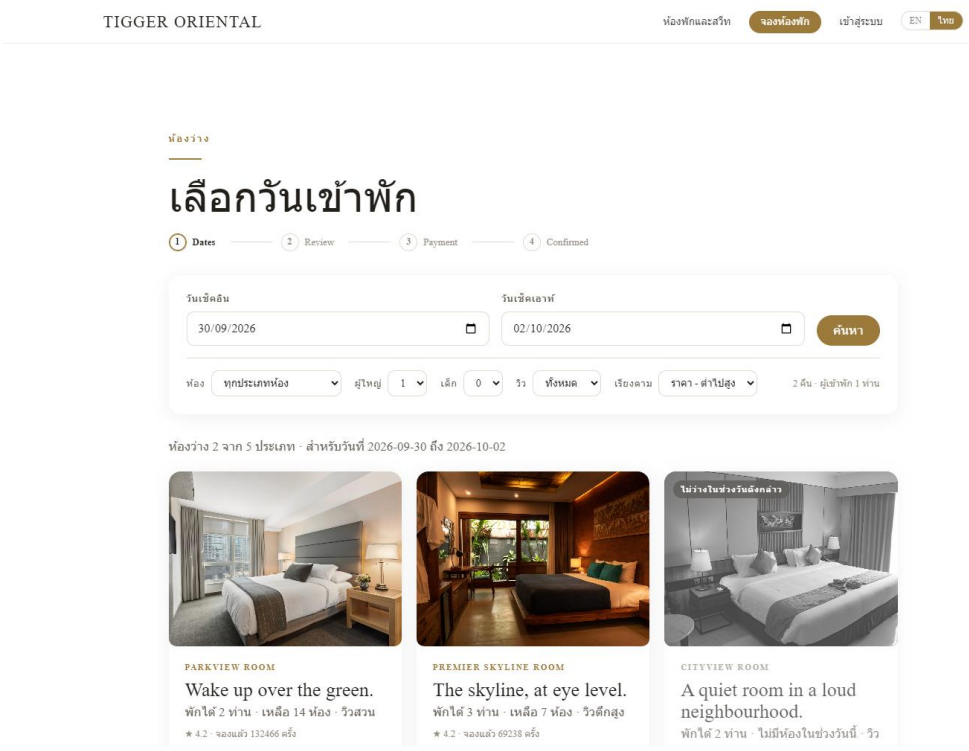
การทดสอบ	**ผล**
ห้ามจองซ้อน (Q13 บนการจองทั้งหมด)	0 แถว ผ่าน
trigger กันจองซ้อน (Q16)	ถูกปฏิเสธ ผ่าน
trigger เช็กเอาต์และ audit (Q22)	ผ่าน
ใบแจ้งหนี้กระทบยอด (Q19)	ตรงทุกใบ ผ่าน
การจองที่เช็กเอาต์แล้วต้องไม่มียอดค้าง (v_booking_bill)	0 รายการที่ค้าง ผ่าน
ชุดทดสอบอัตโนมัติฝั่งเซิร์ฟเวอร์ (pytest)	ผ่าน 198 จาก 198 กรณี
ชุดทดสอบอัตโนมัติฝั่งเว็บ (vitest)	ผ่าน 81 จาก 81 กรณี

บทที่ 8 การนำฐานข้อมูลไปใช้งาน

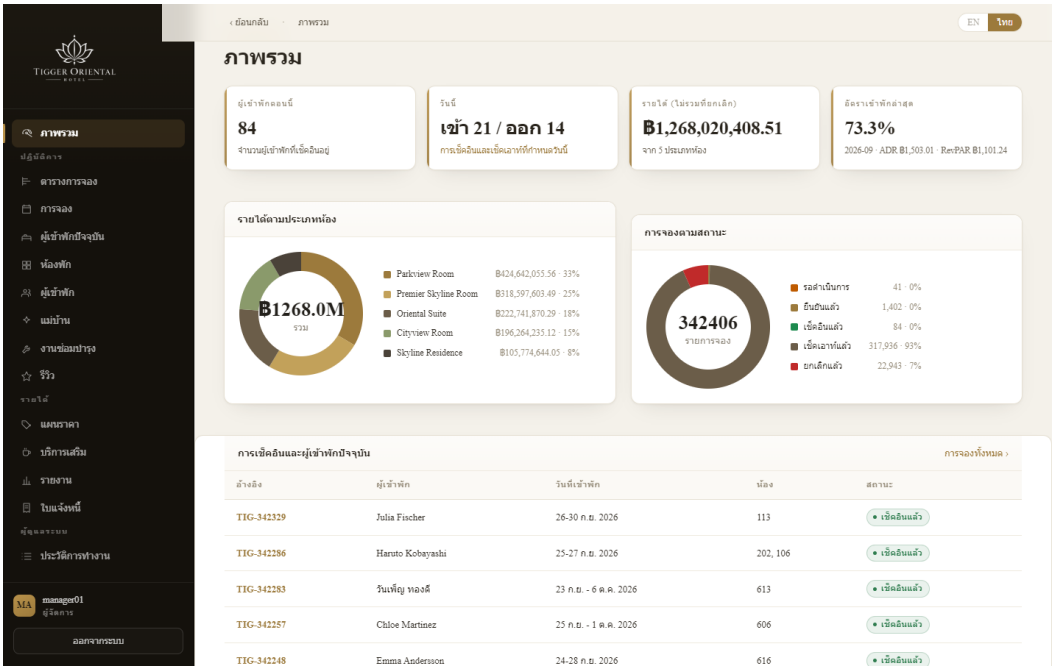
เพื่อยืนยันว่าฐานข้อมูลใช้งานได้จริง กลุ่มได้พัฒนาเว็บแอปพลิเคชันที่เชื่อมต่อฐานข้อมูลนี้ แบ่งเป็นเว็บไซต์สำหรับลูกค้ากับระบบงานสำหรับพนักงาน ทุกตาราง view และ function ในฐานข้อมูลถูกใช้งานผ่านหน้าจอจริง



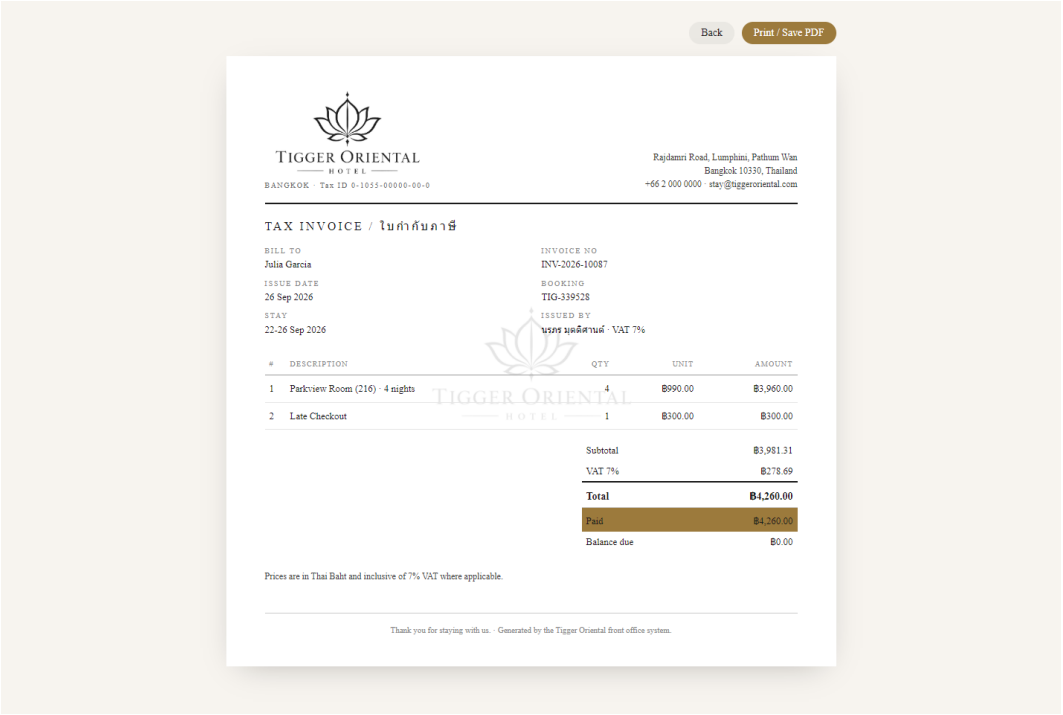
ภาพที่ 8.1 สถาปัตยกรรมของระบบ



ภาพที่ 8.2 หน้าค้นหาห้องว่าง ใช้ fn_available_rooms และ fn_quote_stay



ภาพที่ 8.3 หน้าภาพรวมผู้บริหาร ใช้ v_room_type_revenue



ภาพที่ 8.4 ใบกำกับภาษีจากตาราง invoice และ invoice_line

บทที่ 9 สรุปและข้อเสนอแนะ

9.1 สรุปผล

โครงงานนี้ออกแบบฐานข้อมูลระบบจองห้องพักครบทุกขั้นตอนตามที่เรียนในรายวิชา ผลงานที่ได้มีดังนี้

- ER Diagram 19 entity 25 ความสัมพันธ์
- ฐานข้อมูลในรูปแบบ 3NF ที่สร้างจริงบน PostgreSQL
- constraint 98 รายการ view 7 ตัว function 3 ตัว trigger 6 ตัว
- business query 22 ข้อ ใช้ JOIN หลายตาราง subquery GROUP BY / HAVING รวมถึง window function

ผลการทดสอบบนข้อมูลกว่า 340,000 การจองแสดงว่าไม่มีการจองซ้อน ใบแจ้งหนี้ทั้ง 317,936 ใบมียอดรวมตรงกับบรรทัดรายการ ยอดก่อนภาษีบวก VAT เท่ากับยอดรวมทุกใบ และไม่มีการจองที่เช็กเอาต์แล้วค้างชำระ

9.2 ข้อจำกัด

- กฎลำดับการเปลี่ยนสถานะการจอง รวมถึงกฎการชำระให้ครบก่อนเช็กเอาต์ ยังตรวจที่แอปพลิเคชัน ฐานข้อมูลยังไม่ได้บังคับเอง
- ยังไม่ได้เปิด Row-Level Security การแยกข้อมูลของลูกค้าแต่ละคนทำที่แอปพลิเคชัน
- ถ้าลบการจองด้วยคำสั่ง SQL โดยตรง ข้อมูลการชำระเงินและใบแจ้งหนี้จะถูกลบตามด้วย (ON DELETE CASCADE) ในระบบจึงใช้การยกเลิกแทนการลบ
- ยังไม่ได้ติดตั้งระบบบนคลาวด์

9.3 ข้อเสนอแนะ

- ย้ายกฎลำดับสถานะการจองและการชำระครบก่อนเช็กเอาต์ลงเป็น trigger
- เปิด Row-Level Security ให้ลูกค้าเห็นเฉพาะข้อมูลของตนเอง
- ใช้ materialized view สำหรับรายงานย้อนหลังเมื่อข้อมูลเพิ่มขึ้น
- เปลี่ยน FOREIGN KEY จาก payment และ invoice ไปยัง booking เป็น ON DELETE RESTRICT เพื่อป้องกันการลบหลักฐานทางการเงินโดยไม่ตั้งใจ

9.4 สิ่งที่ได้เรียนรู้

- การออกแบบเป็นขั้นตอน ER ⟶ Relational Schema ⟶ Normalization ช่วยให้ตัดสินใจได้ว่าข้อมูลใดควรแยกตารางและข้อมูลใดควรเก็บซ้ำ
- constraint และ trigger ในฐานข้อมูลช่วยป้องกันข้อมูลผิดได้แม้แอปพลิเคชันจะมีข้อผิดพลาด
- การไม่เก็บค่าที่คำนวณได้ ทำให้ยอดเงินไม่ขัดแย้งกันเอง ยกเว้นยอดในเอกสารภาษีที่ต้องคงค่า